

AI휴먼 — Day 4

Python 프로그래밍

- 조건문으로 프로그램의 흐름 제어하기

오늘의 결과물

조건 기반 성적 판정기

CONDITION

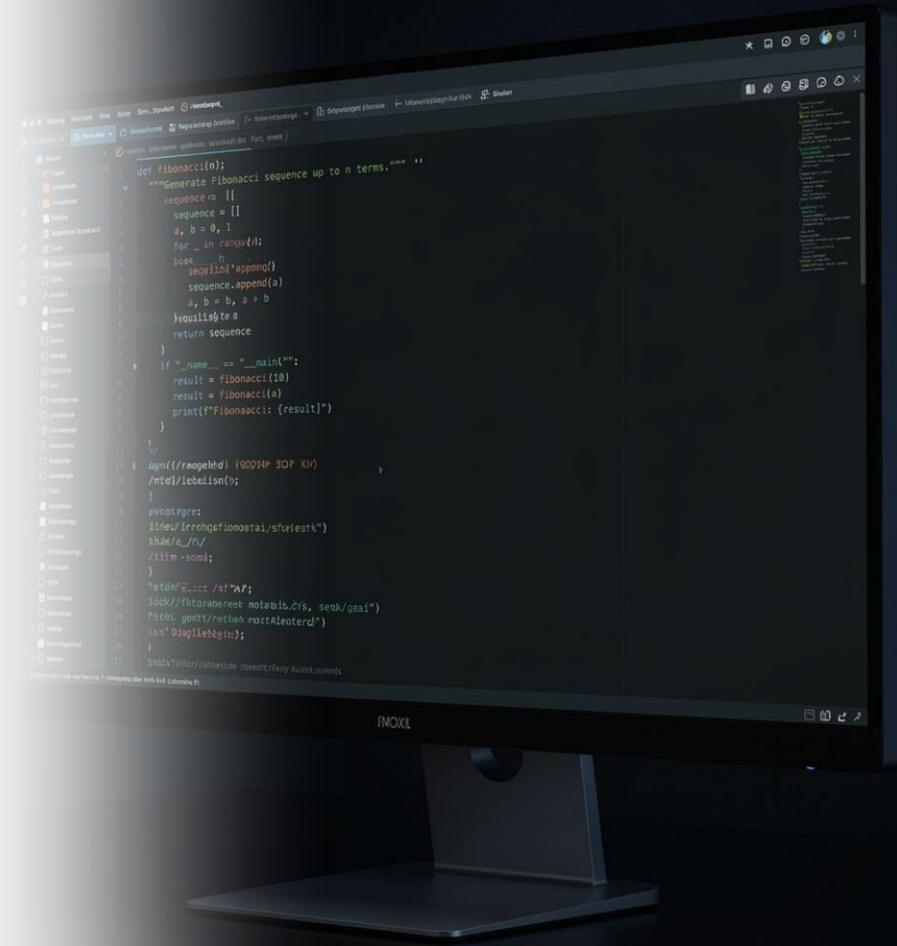
BOOLEAN

IF

ELIF

ELSE

중첩조건



오늘의 학습 목표

- 1 조건식이 True 또는 False로 평가되는 과정을 설명할 수 있습니다.
- 2 if, elif, else를 사용하여 입력값에 따라 서로 다른 처리를 구현할 수 있습니다.
- 3 비교 연산자와 논리 연산자를 조건식에 적용할 수 있습니다.
- 4 여러 조건이 필요한 경우 조건의 순서와 범위를 설계할 수 있습니다.
- 5 점수와 출석률을 이용한 조건 기반 판정 프로그램을 작성할 수 있습니다.

□ 오늘의 학습 흐름: 입력 → 조건 판단 → 분기 처리 → 결과 출력

전날 학습 연결: 입력과 연산

전날 핵심 내용

- `input()`: 사용자 입력 받기
- `int()`, `float()`: 문자열 입력을 숫자로 형변환
- 산술 연산자: `+`, `-`, `*`, `/`, `//`, `%`, `**`
- 비교 연산자: 값의 크기나 같음을 비교
- 논리 연산자: 여러 조건을 결합

예제

```
score = int(input("점수: "))  
print(score >= 80)
```

핵심 연결

오늘은 계산된 값이나 입력값을 '판단'하여
프로그램이 서로 다른 코드를 실행하도록 만듭니다.



프로그램의 흐름과 제어문

Control Flow

프로그램의 명령문이 실행되는 순서

기본 실행

위에서 아래로 한 줄씩 순차 실행

제어문(Control Statement)

조건이나 반복에 따라 실행 흐름을 바꾸는 문장

Python의 대표 제어문

- 조건문: if / elif / else
- 반복문: for / while

오늘은 '조건에 따라 실행 경로를 선택하는 조건문'에 집중합니다.

조건식(Condition)이란?

Condition

참(True) 또는 거짓(False)으로 판단할 수 있는 식

예제

```
score >= 80  
age == 20  
name != "Kim"
```

조건식의 평가 결과

- True: 조건이 성립함
- False: 조건이 성립하지 않음

예제

```
score = 85  
print(score >= 80) # True  
print(score < 60) # False
```

Boolean 자료형

Boolean

참과 거짓을 표현하는 논리 자료형

Python의 Boolean 값

True

False

주의

- 첫 글자는 반드시 대문자
- "True"는 문자열
- True는 Boolean 값

예제

```
is_pass = True  
print(type(is_pass))
```

용어 정리

Boolean: 논리값을 저장하는 자료형

Condition: True/False로 평가되는 조건식

비교 연산자

비교 연산자는 두 값을 비교하여 **Boolean** 결과를 만듭니다.

==

같다

!=

다르다

>

크다

<

작다

>=

크거나 같다

<=

작거나 같다

예제

```
score = 85
```

```
score == 85 → True
```

```
score != 85 → False
```

```
score >= 80 → True
```

```
score < 60 → False
```

주의

= : 값을 저장하는 대입 연산자

== : 두 값이 같은지 비교하는 연산자

논리 연산자

여러 조건을 하나의 조건식으로 결합할 때 사용합니다.

and

모든 조건이 True일 때 True

or

조건 중 하나 이상이 True이면 True

not

True와 False를 반대로 바꿈

예제

```
score = 85
```

```
attendance = 90
```

```
score >= 80 and attendance >= 80 → True and True → True
```

```
score >= 90 or attendance >= 90 → False or True → True
```


if문의 기본 구조

형식

```
if 조건식:  
    실행문
```

동작

1. 조건식을 평가합니다.
2. 결과가 **True**이면 들여쓰기된 코드를 실행합니다.
3. 결과가 **False**이면 해당 블록을 건너뜁니다.

예제

```
score = 85  
  
if score >= 80:  
    print("PASS")  
  
print("판정 완료")
```

출력

```
PASS  
판정 완료
```

의사코드: 단일 조건 판단

의사코드(Pseudocode)

실제 프로그래밍 언어의 문법보다 문제 해결 절차를
사람이 이해하기 쉽게 표현한 코드

문제

점수가 80점 이상이면 PASS 출력

의사코드

점수를 입력받는다
만약 점수가 80 이상이면 "PASS"를 출력한다
"판정 완료"를 출력한다

Python

```
score = int(input("점수: "))

if score >= 80:
    print("PASS")

print("판정 완료")
```

들여쓰기(Indentation)

Python은 들여쓰기로 코드 블록을 구분합니다.

올바른 코드

```
if score >= 80:
    print("PASS")
    print("축하합니다.")
```

잘못된 코드

```
if score >= 80:
print("PASS")
```

핵심 규칙

- if문 끝에는 콜론(:)
- 조건에 포함되는 코드는 들여쓰기
- 같은 블록의 문장은 같은 깊이로 들여쓰기
- 일반적으로 공백 4칸 사용

⚠ 들여쓰기는 단순한 보기 좋기 규칙이 아니라 Python 문법입니다.

if - else

두 가지 경우 중 하나를 선택할 때 사용합니다.

형식

```
if 조건식:  
    조건이 True일 때 실행  
else:  
    조건이 False일 때 실행
```

예제

```
score = 75  
  
if score >= 80:  
    print("PASS")  
else:  
    print("RETRY")
```

출력

RETRY

else에는 조건식을 작성하지 않습니다.

if - elif - else

세 가지 이상의 경우를 구분할 때 사용합니다.

형식

```
if 조건1:
    실행문1
elif 조건2:
    실행문2
elif 조건3:
    실행문3
else:
    나머지 실행문
```

특징

- 위에서 아래로 조건을 확인
- 처음 **True**가 된 블록 하나만 실행
- 이후 조건은 검사하지 않음

📌 따라서 조건의 '순서'가 매우 중요합니다.

성적 등급 판정 예제

```
score = 85

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "F"

print(grade)
```

실행 과정

1. $85 \geq 90 \rightarrow \text{False}$
2. $85 \geq 80 \rightarrow \text{True}$
3. `grade = "B"`
4. 이후 조건은 검사하지 않음

결과

B

조건의 순서가 중요한 이유

잘못된 순서

```
score = 95

if score >= 60:
    grade = "D"
elif score >= 70:
    grade = "C"
elif score >= 80:
    grade = "B"
elif score >= 90:
    grade = "A"
```

결과

D

이유

첫 번째 조건 `score >= 60`이 이미 True이기 때문

개선

범위가 좁거나 기준이 높은 조건부터 검사



범위 조건 표현하기

방법 1: 논리 연산자 사용

```
score >= 80 and score < 90
```

방법 2: Python 비교 연산 연결

```
80 <= score < 90
```

두 조건식은 같은 의미입니다.

예제

```
score = 85

if 80 <= score < 90:
    print("B 구간")
```

실무에서 중요한 점

조건 범위가 겹치거나 빠지지 않도록 경계값을 확인해야 합니다.

경계값 예

79, 80, 89, 90

여러 조건을 함께 판단하기

문제

점수가 70점 이상이고 출석률이 80% 이상이면 합격

조건

- `score >= 70`
- `attendance >= 80`

결합

```
score >= 70 and attendance >= 80
```

Python

```
if score >= 70 and attendance >= 80:  
    print("PASS")  
else:  
    print("FAIL")
```

`and`는 '둘 다 만족'해야 하는 업무 규칙에 자주 사용됩니다.

or와 not 활용

or 예제

온라인 또는 오프라인 중 하나만 신청해도 수강 가능

```
mode = "online"

if mode == "online" or mode == "offline":
    print("신청 가능")
```

not 예제

로그인하지 않은 경우 안내

```
is_login = False

if not is_login:
    print("로그인이 필요합니다.")
```

읽는 방법

not is_login → is_login이 아니다 → 로그인 상태가 False이면 True

중첩 조건문(Nested if)

조건문 안에 다른 조건문을 작성할 수 있습니다.

예제

```
score = 85
attendance = 75

if score >= 70:
    if attendance >= 80:
        print("PASS")
    else:
        print("FAIL: 출석률 부족")
else:
    print("FAIL: 점수 부족")
```

장점

판정 과정을 단계별로 표현하기 쉬움

주의

중첩이 너무 깊으면 코드 이해가 어려워질 수 있습니다.

input()과 조건문 연결

input()의 결과는 문자열입니다.

잘못된 예

```
score = input("점수: ")

if score >= 80:
    print("PASS")
```

문제

문자열과 정수를 직접 비교할 수 없음

개선

```
score = int(input("점수: "))

if score >= 80:
    print("PASS")
```

입력 → 형변환 → 조건 판단 → 출력 이 흐름을 습관적으로 확인합니다.

유효한 입력 범위도 조건으로 검사하기

점수의 정상 범위: 0 ~ 100

```
score = int(input("점수: "))

if score < 0 or score > 100:
    print("입력 오류")
elif score >= 80:
    print("PASS")
else:
    print("RETRY")
```

핵심

조건문은 업무 규칙뿐 아니라 입력 데이터가 정상 범위인지

검사하는 데도 사용할 수 있습니다.

대표 실습 설계: 조건 기반 성적 판정기

입력

- 점수(score)
- 출석률(attendance)

처리

1. 입력 범위가 정상인지 검사
2. 점수에 따라 등급 판정
3. 점수와 출석률을 함께 이용해 최종 합격 여부 판정

출력

- 등급
- 최종 판정
- 판정 이유

학습 포인트

비교 연산자

논리 연산자

if / elif / else

중첩조건

경계값

대표 실습 의사코드

점수를 입력받는다

출석률을 입력받는다

만약 점수가 0보다 작거나 100보다 크면

"점수 입력 오류"를 출력한다

그렇지 않고 출석률이 0보다 작거나 100보다 크면

"출석률 입력 오류"를 출력한다

그렇지 않으면

점수가 90 이상이면 A

아니고 80 이상이면 B

아니고 70 이상이면 C

아니고 60 이상이면 D

그 외에는 F

만약 점수가 70 이상이고 출석률이 80 이상이면

"PASS" 출력

그렇지 않으면

"FAIL" 출력

대표 실습 코드 핵심

```
score = int(input("점수: "))
attendance = int(input("출석률(%): "))

if score < 0 or score > 100:
    print("점수 입력 오류")
elif attendance < 0 or attendance > 100:
    print("출석률 입력 오류")
else:
    if score >= 90:
        grade = "A"
    elif score >= 80:
        grade = "B"
    elif score >= 70:
        grade = "C"
    elif score >= 60:
        grade = "D"
    else:
        grade = "F"

    print(f"등급: {grade}")
```


Basic 과제

목표

점수 한 개를 입력받아 합격 여부 출력

규칙

- 60점 이상: PASS
- 60점 미만: RETRY

입력 예

점수: 75

출력 예

PASS

필수 요소

input()

int()

if

else

확인할 경계값

59, 60, 61

Application 과제

목표

점수 구간에 따라 A~F 등급 판정

규칙

- 90~100: A
- 80~89: B
- 70~79: C
- 60~69: D
- 0~59: F

추가 규칙

0보다 작거나 100보다 큰 점수는 "입력 오류"

핵심 포인트

- 높은 기준부터 비교
- `elif` 사용
- 경계값 확인

Challenge 과제

목표

점수와 출석률을 함께 사용하여 최종 판정

입력

- 점수
- 출석률

규칙

- 범위 오류가 있으면 입력 오류
- 점수 90 이상 AND 출석률 95 이상: 우수 합격
- 점수 70 이상 AND 출석률 80 이상: 합격
- 출석률 80 미만: 출석 미달
- 그 외: 점수 미달

핵심 포인트

- and
- elif의 순서
- 여러 업무 규칙의 우선순위

자주 발생하는 오류와 디버깅

오류 1

if score = 80: → 비교는 == 사용

오류 2

if score >= 80 → 콜론(:) 누락

오류 3

input() 결과를 숫자로 변환하지 않음 →
int(input(...))

오류 4

조건 순서가 잘못됨 → 높은 기준부터 확인

오류 5

경계값 누락 → 59/60, 79/80, 89/90처럼
경계값 테스트

디버깅 질문

1. 현재 변수 값은 무엇인가?
2. 조건식 결과는 True인가 False인가?
3. 어떤 블록이 실행되어야 하는가?

오늘의 핵심 정리와 다음 학습

오늘의 핵심

- Condition: True/False로 평가되는 식
- Boolean: True와 False를 표현하는 자료형
- if: 조건이 True일 때 실행
- elif: 추가 조건 검사
- else: 앞 조건이 모두 False일 때 실행
- and/or/not: 여러 조건 결합
- 중첩조건: 조건 안에서 추가 조건 판단

문제 해결 순서



다음 학습

for / while / range / list / dictionary

여러 데이터를 반복해서 처리하는 방법

